# Conversational SHL Assessment Recommender

## Design Choices

- FastAPI exposes a stateless `GET /health` and `POST /chat`; every request is reconstructed from the supplied message history.
- The scraped SHL Individual Test Solutions catalog in `data/catalog.json` is the source of truth. Pre-packaged Job Solutions are excluded by crawling only catalog `type=1`.
- The app builds an in-memory deterministic retriever at startup. It never scrapes during request handling.
- LLM generation is optional. Without keys, deterministic templates still clarify, recommend, refine, compare, and refuse.
- Final output is validated through Pydantic plus catalog-only URL repair. Recommendations can only be records from the local catalog.

## Retrieval Setup

- Indexed fields: name, description, raw detail text, test type, job levels, languages, duration, remote testing, adaptive/IRT, and keywords.
- Lexical retrieval combines word TF-IDF with character n-grams so terms like Java, C++, .NET, OPQ, GSA, and product variants still match.
- Optional semantic retrieval is supported if sentence-transformers is installed, but the deployable baseline stays lightweight.
- Rule boosts cover exact skills, assessment families, role/seniority context, requested test types, and constraints such as no personality or short tests.
- Diversity selection prevents the shortlist from becoming only near-duplicate variants, while still preserving explicit constraints like Java plus personality.

## Prompt Design

- The LLM receives only a conversation summary and retrieved catalog evidence; it is not allowed to invent names, URLs, test types, durations, or capabilities.
- Deterministic code extracts constraints, retrieves candidates, validates final recommendations, and handles refusals before any LLM text is trusted.
- Clarification asks one high-value question only when there is no role, skill, assessment family, job description, or test-type clue.
- Comparison responses are grounded in catalog fields and explicitly say when duration, remote testing, adaptive/IRT, or job-level metadata is missing.

## Evaluation Method

- `scripts/run_eval.py` measures schema compliance, catalog validity, Recall@10, lexical relevance, approximate groundedness, and behavior probe pass rate.
- Behavior probes cover vague clarification, recommendation, refinement, comparison, off-topic refusal, prompt-injection refusal, URL hallucination, and max-10 limits.
- Recall@10 uses labeled expected catalog names/URLs where available; relevance and groundedness use local catalog evidence, not model self-judgment.

## What Did Not Work

- Pure LLM recommendations hallucinated assessment names and URLs, so the model was removed from the source-of-truth path.
- Keyword-only retrieval missed synonyms and job-description phrasing; adding character n-grams and rule boosts improved technical and acronym matching.
- Asking several clarification questions hurt the eight-turn evaluator cap; one concise question worked better.
- Long prompts increased latency and did not improve recall enough to justify using them on every request.

## How Improvement Was Measured

- Baseline keyword retrieval was compared with hybrid lexical retrieval plus rule boosts on Java, Python, sales, contact center, graduate, leadership, OPQ, and Excel cases.
- Current metrics are written to `data/evaluation/results.json`: Recall@10, catalog validity, schema compliance, behavior pass rate, relevance, and groundedness.
- LLM used: configurable through `LLM_PROVIDER` and `MODEL_NAME`; deployment defaults to Gemini 2.5 Flash when `GEMINI_API_KEY` is set.
- AI tools used: Codex for implementation assistance, with catalog checks, tests, and evaluation used as review gates.